Relatório

Criptografia Aplicada Avançada 2026

Project Assignment 1

Prof. Henrique João Lopes Domingos

Diogo Nunes 70502

Pedro Galego 72298



Introdução

O objetivo deste projeto foi desenvolver o *design* e implementação de um canal de UDP seguro para suportar *streaming* em tempo real baseado num protocolo de partilha de dados com proteção criptográfica, nomeadamente em encriptação e controlo de integridade de pacotes com *media frames* para filmes em MPEG4.

Dado que o protocolo de transporte UDP não garante ordem, integridade, confidencialidade nem proteção contra *Replay Attacks*, o desafio do projeto consistia em garantir todas estas propriedades de segurança como camada extra de proteção.

Como início, foram implementados componentes para auxiliar na realização do projeto:

- *hjStreamServer*
    - Servidor responsável por ler os *frames* do filme escolhido, cifrar os pacotes e enviá-los;
    - Permite ao utilizador escolher entre duas cifras para encriptar os pacotes UDP;
    - Tem acesso a uma pasta com dois filmes:
        - cars.dat;
        - monsters.dat.

- *hjUDPProxy*
    - *Proxy* usado como interface entre o utilizador e o servidor;
    - Responsável por receber os pacotes UDP cifrados pelo servidor, descartando pacotes fora de ordem ou repetidos;
    - Decifra os pacotes UDP recebidos e encaminha-os para o endereço definido pelo utilizador.

Segurança

De forma a completar os requisitos do projeto, foram implementados vários mecanismos de segurança para garantir a troca segura de pacotes UDP entre o servidor e o *proxy*.

Para garantir confidencialidade, foi implementado mecanismos de encriptação e desencriptação, com várias *ciphersuites* suportadas, como explicado no enunciado:

- AES/CTR/NoPadding + HMAC-SHA256;
- AES/CBC/PKCS5Padding + HMAC-SHA256;
- AES/GCM/NoPadding;
- ChaCha20-Poly1305.

Já dentro da parte dois do projeto, é utilizado *Elliptic Curve Diffie-Hellman* para a partilha de segredos entre o servidor e o *proxy* para derivar chaves de sessão, de forma a assegurar um canal inicial seguro de comunicação entre eles.

Os modos AES/CTR e AES/CBC garantem apenas confidencialidade, sendo necessário recorrer a mecanismos adicionais de autenticação e controlo de integridade. Para tal, foi utilizado o HMAC-SHA256, permitindo validar a autenticidade e integridade dos pacotes transmitidos. Em contraste, AES/GCM e ChaCha20-Poly1305 pertencem à categoria de modos AEAD (Authenticated Encryptionwith Associated Data), fornecendo simultaneamente confidencialidade, integridade e autenticação. Para além disto, o *proxy* e o servidor autenticam-se durante o *handshake* através de certificados X.509 e assinaturas digitais.

A proteção contra *Replay Attacks* é feita a partir de um número de sequência (*seqNum*) único a cada pacote, incluído no *header* de cada um. Caso algum destes pacotes verifique um número de sequência inferior ao recebido em último lugar é descartado pelo *proxy*, mitigando assim este tipo de ataques.

Secure Handshake Protocol

O protocolo SHP implementado permite que o servidor e o *proxy* estabeleçam uma conexão segura através de derivação de chaves a partir de um segredo comum. Para além do já explicado anteriormente, este protocolo também tem proteção contra *replay attacks* através de *nonces* atribuídos a

ambas as entidades no momento da ligação, que são renovados em todas as ligações.

O protocolo é iniciado com uma mensagem de *ClientHello*, com a escolha do filme, o certificado X.509 do *proxy*, a chave pública ECDH, um *nonce* aleatório, a lista de *ciphersuites* suportadas e a assinatura digital sobre os dados, para garantir autenticidade e integridade. Após a validação da mensagem de início, o servidor responde com *ServerHello*, com os mesmos atributos do *ClientHello* mas referentes ao servidor (à exceção da *ciphersuite* selecionada, ao invés da lista das diferentes opções). Após validação da resposta do servidor por parte do *proxy* (*truststore* do certificado, assinatura digital, resposta correta ao nonce enviado e a validade da *ciphersuite* escolhida) é realizada a derivação de uma chave de encriptação e desencriptação e uma chave MAC para cada parte da comunicação, a partir do segredo partilhado ECDH.

Implementação

Na primeira parte foi implementado:

- O envio dos pacotes UDP entre o servidor e o *proxy*;
- A encriptação e desencriptação dos pacotes por parte do servidor e do *proxy*, respetivamente;
- Suporte para alternativa de *ciphersuites*;
- Mitigação de *replay attacks* a partir de número de sequência no *header* dos pacotes UDP e respetiva validação.

Na segunda parte foi acrescentado:

- O protocolo SHP;
- Criação e utilização de keystores e truststores para autenticação mútua entre cliente e servidor;
- Geração e validação de certificados digitais auto-assinados utilizados durante o processo de handshake.
- Criação de canal de comunicação seguro inicial para troca de chaves via ECDH;
- Derivação de chaves de sessão a partir do segredo partilhado;
- Proteção contra *replay* durante o *handshake* inicial.

Testes e Resultados

A partir de execuções feitas com mostra de dados essenciais nos protocolos na consola, foi possível observar a contagem correta dos *frames* enviados e receção contínua dos pacotes UDP até ao final do conteúdo, com derivação semelhante de chaves em ambas as partes do diálogo.

Os testes foram realizados para todos os casos possíveis de *input* do utilizador, obtendo os seguintes resultados:

- cars.dat:
    - AES/CTR/NoPadding:
        - Total frames sent: 30515;
        - Movie duration 128 s;
        - Throughput 238 fps;
        - Throughput 2509 Kbps.
    - AES/CBC/PKCS5Padding:
        - Total frames sent: 30515;
        - Movie duration 128 s;
        - Throughput 238 fps;
        - Throughput 2509 Kbps.
- monsters.dat:
    - ChaCha20-Poly1305:
        - Total frames sent: 25945;
        - Movie duration 141 s;
        - Throughput 184 fps;
        - Throughput 1937 Kbps.
    - AES/GCM/NoPadding:
        - Total frames sent: 25945;
        - Movie duration 141 s;
        - Throughput 184 fps;
        - Throughput 1937 Kbps.